

Final Nm-index (Nilmantha's index)

Implementation summary & assumptions for supervisor review

Module: **final_nm_index/** | Repo: Software-Group-Project-Bibliometric-Data-Collection | Date: 2026-09-04

This module implements **only the last stage** of the Nm-index model — Equations 24–27 of the full paper (§2.5 “Final score”; Eq. 23–25 in the extended abstract). Everything before Eq. 24 (author-contribution weights, influential self-citation, career factor, and the six component metrics themselves) is produced by other pipelines in the repo and stored in PostgreSQL. This module reads those stored values and does **not** recompute them.

1. What the code does

For every author it turns six raw impact numbers into one comparable percentile score:

#	Symbol	Meaning	Paper Eq.	Read from (DB)
x1	T_a	Outlier-uncontrolled field- & year-normalised total cites	Eq. 11	author_total_cites.total_cites_score
x2	S_a	Outlier-controlled field-normalised cites per paper	Eq. 13	author_citations_per_paper.citations_per_paper_score
x3	U_a	Outlier-controlled field- & year-normalised citation rate	Eq. 14	author_citation_rate.citation_rate_score
x4	Hf'_a	Modified hf-index	Eq. 17	author_modified_hindex.modified_hindex_final
x5	Hm'_a	Modified hm-index	Eq. 20	author.modified_hm_index
x6	G'_a	Modified g-index	Eq. 23	author.modified_g_index

Processing chain:

- **Load** (run_calculation.py) — one **LEFT JOIN** of the six one-row-per-author tables onto the full author list, keyed on author_id. Only rows with a non-NULL value are taken for each metric.
- **Calculate** (calculator.py) — Eq. 24 transform then Eq. 25 normalise then Eq. 26 percentile, per metric; then Eq. 27 weighted average. Pure pandas / numpy / scipy, no DB access.
- **Write** (run_calculation.py) — UPSERT one row per author into **author_nm_index** (the six percentile columns, metrics_available, nm_index, calculated_at). Optionally mirror nm_index onto the author table. Re-running is idempotent.

2. How it implements Equations 24–27

The six metrics split into two families that the paper treats differently.

2.1 Citation metrics — T_a, S_a, U_a

- **Eq. 24 (transform):** $L(x) = \log_{10}(x + 1)$. The +1 keeps $\log(0)$ defined; the log compresses the heavy right tail.
- **Eq. 25 (normalise), default ‘hooked power law’:** $\text{Nor}(x) = \Phi^{-1}(P(L(x)))$, where $P(v) = (r(v) - 0.375) / (N + 0.25)$ is the Blom plotting position on the rank r , and Φ^{-1} is the inverse standard-normal CDF.
- **Eq. 25 alternative ‘log-normal’:** $\text{Nor}(x) = (L(x) - \text{mean } L(x)) / \text{sd } L(x)$ (population sd).
- **Eq. 26 (percentile):** $\text{Per}(x) = 100 \cdot \Phi(\text{Nor}(x))$.

Exact simplification used in code: for the hooked branch, $100 \cdot \Phi(\Phi^{-1}(P)) = 100 \cdot P$ identically, so the code computes $\text{Per}(x) = 100 \cdot P(L(x))$ directly and never forms the intermediate Nor. Eq. 27 only needs Per, so nothing is lost.

2.2 Index metrics — Hf'_a, Hm'_a, G'_a

- **Eq. 24 (transform):** $P(x) = (r(x) - 0.375) / (N + 0.25)$ directly on the raw index, with **fractional ranking** (tied values share the mean rank — essential because many authors have $Hm' = Hf' = G' = 0$).
- **Eq. 25:** $Nor(x) = \Phi^{-1}(P(x))$.
- **Eq. 26:** $Per(x) = 100 \cdot P(x)$.

2.3 Ranking method (per Eq. 24 wording)

Family	Paper wording	Code setting	pandas method
Hf, Hm', G'	"fractional ranking ... (many ties)"	RANK_METHOD_RANK	"average"
T, S, U	"normal ranking for the raw citations"	RANK_METHOD_LOG	"min" (competition)

2.4 Eq. 27 — final score

$Nm_a = (i=1..6) w_i \cdot Per_a(x_i)$, with $\sum w_i = 1$. Default $w_i = 1/6$ (equal weighting, the paper's suggestion) then a plain average of the six percentiles.

3. Alignment with the paper's worked example (§3.3.3)

§3.3.3 “Final Nm-Index Calculation” is the paper's only end-to-end numerical example. It states the exact recipe the code follows:

“citation metrics (T_a , S_a , U_a) follow a hooked power law, so $Nor_a(x) = \Phi^{-1}(P(L(x)))$... normal ranking. Indices (Hf' , Hm' , G') use fractional ranking for ties, with $Nor_a(x) = \Phi^{-1}(P(x))$.” Then $Nm_a = (1/6) \sum Per$.

Example metric	Raw	$L(x)=\log_{10}(x+1)$	P (from rank)	$Per(x)=100 \cdot P$
T_a	1.683	0.4286	0.5500	55.00
S_a	0.374	0.1380	0.3000	30.00
U_a	0.629	0.2119	0.4500	45.00
Hf'_a	0.415	— (rank raw)	0.4000	40.00
Hm'_a	0.113	— (rank raw)	0.2500	25.00
G'_a	0.365	— (rank raw)	0.3500	35.00

$Nm_a = (55 + 30 + 45 + 40 + 25 + 35) / 6 \sim \mathbf{38.33}$ (~ 38th percentile globally).

Not reproducible exactly, and this is expected: the P values above come from ranks the paper asserts by hand (“r ~ 55000 (mid-range)” inside a hypothetical $N = 100,000$ sample), not from a real dataset. The code reproduces the **formula path** that generates them; the specific figures are illustrative only. A unit test (test_matches_paper_section_333_mechanism) verifies the mechanism on a controlled input.

4. Assumptions & choices requiring your sign-off

Each of these is a point where the paper is silent, ambiguous, or assumes ideal data. All are isolated in **config.py** and switchable without touching the maths. Current defaults are stated; please confirm or redirect.

A. Distribution assumption for T_a , S_a , U_a — NM_DISTRIBUTION

Eq. 25 branches on whether a metric “follows log normal” (Z-score) or “follows hooked power law” (rank-based inverse normal). §2.5 says this varies by research stream (humanities / social science ~ log-normal; medical / natural science ~ hooked).

Assumed: “hooked” for all three metrics — matches the §3.3.3 worked example, and by the time these metrics reach this module they are already cross-field per-author aggregates, so there is no single field left to branch on. Switch any metric to “lognormal” if that stream is genuinely log-normal.

B. Reference population N in $P(x) = (r - 0.375)/(N + 0.25)$ — NM_REFERENCE_N

§3.3.3 uses $N = 100,000$ because it ranks one profile inside a global 100k-profile sample.

Assumed: N = number of authors actually scored for that metric (currently ~2,300), i.e. $NM_REFERENCE_N = None$. Percentiles are therefore **relative to the loaded cohort**, not a global figure. A fixed $N = 100,000$ on a 2,300-author cohort would push every percentile toward 0, so cohort- N is the correct choice at this data scale — but the output should be read as “percentile within this dataset”.

C. Missing / NULL component metric — NM_REQUIRE_ALL_METRICS

The paper assumes all six metrics exist for every author. In the real DB some are NULL (notably total_cites: only 1561 / 2507 authors).

Assumed: $NM_REQUIRE_ALL_METRICS = False$. An author missing k of the six is scored on the remaining $6-k$, with the weights renormalised to sum to 1 over the available subset (so with equal weights, nm_index is the plain mean of the percentiles the author has). A NULL is **never** treated as a zero — it is dropped from both the ranking population and the average. The metrics_available column (0–6) records how many were used. Set to True to instead output NULL for any author without all six.

D. Rank tie method for T_a , S_a , U_a — RANK_METHOD_LOG

Eq. 24 says “normal ranking for the raw citations”. Assumed = “min” (competition ranking). **Consequence on real data:** the citation metrics also contain a large block of exact 0.0; under “min” that whole block shares rank 1 and lands near the 0th percentile. Switch to “average” to spread the zero-block to mid-scale instead.

E. Other minor, low-risk choices

- Population standard deviation (ddof = 0) in the log-normal branch — the natural reading of “standard deviation”.
- `total_cites` is read from `author_total_cites` (identical twin of `equation_11_total_cites`); switch in `config.METRIC_SOURCES` if desired.
- The unreliable `calculation_complete` / `calculation_status` flags on the source tables are ignored by default (filter on value IS NOT NULL only); `NM_FILTER_STATUS = True` re-enables them.
- `G'_a` is read from `author.modified_g_index` (the `modified_g_index_results` table exists but is empty).

5. Current data state (DB checked 2026-09-04, read-only)

Component metric	Authors with a value	Note
total_cites (T_a)	1561 / 2507	rest still NULL upstream
citations_per_paper (S_a)	2321 / 2507	
citation_rate (U_a)	2321 / 2507	
modified_hf_index (Hf'_a)	2143 / 2507	197 authors NO_ELIGIBLE_FIELDS
modified_hm_index (Hm'_a)	2254 / 2507	
modified_g_index (G'_a)	2143 / 2507	on author.modified_g_index

Dry-run result (calculation only, nothing written):

metrics_available	6	5	4	3	2	0
authors	1537	606	24	87	67	186

Scored: 2321 / 2507 authors. nm_index — median ~ 24.6, mean ~ 27.1, max ~ 99.95. The low median is a direct symptom of the **upstream component metrics being ~96% exact zero**: most authors tie at the bottom of each ranking, with a thin tail of genuine achievers near 100. This is mathematically consistent with the model, not a code defect, but the distribution shape is worth discussing.

6. Output

Table **author_nm_index** (created by schema.sql), one row per author:

Column	Meaning
author_id	PK, FK -> author
per_total_cites ... per_modified_g_index	the six Per_a(x_i) values, 0–100, NULL if that component is unavailable
metrics_available	SMALLINT 0–6 — how many components fed nm_index
nm_index	final Eq. 27 score; NULL if 0 components (or <6 when NM_REQUIRE_ALL_METRICS)
calculated_at	run timestamp

Optionally (NM_UPDATE_AUTHOR_TABLE = True) the final score is also mirrored onto author.nm_index, the same way author.modified_hm_index is populated.

7. Verification status

- **13 unit tests pass** (test_calculator.py, pure in-memory, no DB). Every expected value is hand-derived in the test comments; Phi is taken from scipy.stats.norm exactly as Eq. 25/26 intend.
- Tests cover: Blom plotting position on fractional ranks; log + Z-score branch; log + hooked branch; the §3.3.3 mechanism on a controlled 5-author input; weight renormalisation over missing metrics; NM_REQUIRE_ALL_METRICS; all-NULL author -> NULL; custom weights; and the NM_REFERENCE_N override.
- **DB-to-calculator path** validated by read-only dry runs against the live database. No automated test exercises run_calculation.py / database.py (same known gap as the modified_hm_index module).
- The maths of Eq. 24–27 is implemented faithfully. The three items in section 4 (A–C) are modelling choices, not bugs; they are documented, defaulted sensibly, and configurable.

Appendix — config.py knobs

Setting	Default	Effect
NM_WEIGHTS	1/6 each	Eq. 27 weights w_i (must sum to 1)
NM_DISTRIBUTION	"hooked" x3	per-citation-metric: "hooked" or "lognormal"
NM_REFERENCE_N	None	None = cohort size; int = fixed N in P(x)
RANK_METHOD_LOG	"min"	tie handling for T/S/U ranking
RANK_METHOD_RANK	"average"	tie handling for Hf/Hm'/G' ranking (fractional)
NM_REQUIRE_ALL_METRICS	False	False = score on available subset; True = NULL unless all 6
NM_FILTER_STATUS	False	also require calculation_complete / status = READY
NM_UPDATE_AUTHOR_TABLE	False	also write author.nm_index
BLOM_A	0.375	Blom constant a in $(r - a)/(N + 1 - 2a)$